Simulation Mode Guide

This guide explains how to use the Franka User Control system in simulation mode without requiring a real robot.

Overview

Simulation mode allows you to:

- Test your code without robot hardware
- Learn the API safely
- Develop and debug control sequences
- Validate trajectories before running on real hardware

Quick Start

1. Launch Simulation

```bash
# Source your workspace
source ~/franka_user_control_ws/install/setup.bash

# Launch simulation (without RViz)
ros2 launch franka_user_control simulation.launch.py use_rviz:=false

# Or with RViz for visualization
ros2 launch franka_user_control simulation.launch.py use_rviz:=true
```

2. Run Test Scripts

In a new terminal:

```bash
source ~/franka_user_control_ws/install/setup.bash

# Run simulation test
ros2 run franka_user_control test_simulation.py
```

3. Use Python API

The Python API works exactly the same in simulation:

```python
from franka_python_interface import FrankaInterface

# Create interface (will detect simulation mode automatically)
robot = FrankaInterface()
robot.connect()

# Use normal commands
robot.set_cartesian_velocity(vx=0.05, duration=2.0)
robot.move_relative(dx=0.1, dy=0.0, dz=0.0)

robot.disconnect()
```

Simulation Components

Fake Robot State Publisher

Publishes simulated joint states at 100 Hz. The simulated robot:

- Starts at home position
- Smoothly interpolates between positions
- Responds to velocity commands
- Can optionally add noise for realism

Simulation Bridge Node

Provides the same ROS2 services as the real bridge:

- Accepts velocity commands
- Simulates motion with realistic timing
- Publishes current pose
- Tracks robot state

Differences from Real Robot

Feature	Real Robot	Simulation
Hardware required	Yes	No
Safety reflexes	Active	Simulated
Force feedback	Available	Not simulated

Feature	Real Robot	Simulation
Timing accuracy	1 kHz	~100 Hz
Collision detection	Hardware-level	Not implemented
Position accuracy	High	Perfect (no noise)

Testing Workflow

Recommended workflow for development:

1. **Develop in simulation**

```bash
# Test basic functionality
ros2 launch franka_user_control simulation.launch.py
# Run your test scripts
```

2. **Validate trajectories**
   - Check for workspace violations
   - Verify velocity limits
   - Test error handling
3. **Transfer to real robot**

```bash
# Switch to real hardware
ros2 launch franka_user_control user_control.launch.py robot_ip:=YOUR_IP
# Run same test scripts with conservative limits
```

Simulation Parameters

Configure simulation behavior via launch arguments:

```bash
```

```
# Adjust publishing rate
ros2 launch franka_user_control simulation.launch.py publish_rate:=50

# Enable conservative limits
ros2 launch franka_user_control simulation.launch.py conservative_limits:=true

# Set logging level
ros2 launch franka_user_control simulation.launch.py log_level:=debug
```

Monitoring Simulation

Check Joint States

bash

```
# Echo joint states
ros2 topic echo /joint_states

# Check publishing rate
ros2 topic hz /joint_states
```

Monitor Robot Pose

bash

```
# Echo current pose
ros2 topic echo /current_pose
```

View Topics

bash

```
# List all active topics
ros2 topic list

# Get topic info
ros2 topic info /joint_states
```

Example Scripts

Basic Velocity Test

python

```python
#!/usr/bin/env python3
import rclpy
from geometry_msgs.msg import Twist
from rclpy.node import Node

rclpy.init()
node = Node('test_node')

pub = node.create_publisher(Twist, 'velocity_command', 10)

# Send velocity command
msg = Twist()
msg.linear.x = 0.05  # 5 cm/s
pub.publish(msg)

# ... wait and stop
msg.linear.x = 0.0
pub.publish(msg)

rclpy.shutdown()
```

Position Monitoring

python

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Pose

class PoseMonitor(Node):
    def __init__(self):
        super().__init__('pose_monitor')
        self.sub = self.create_subscription(
            Pose, 'current_pose', self.callback, 10)

    def callback(self, msg):
        print(f"Position: [{msg.position.x:.3f}, "
              f"{msg.position.y:.3f}, {msg.position.z:.3f}]")

rclpy.init()
node = PoseMonitor()
rclpy.spin(node)
rclpy.shutdown()
```

Troubleshooting

Simulation not starting

Problem: Nodes don't start **Solution**:

```bash
# Check if ROS2 is sourced
source /opt/ros/humble/setup.bash
source ~/franka_user_control_ws/install/setup.bash

# Verify package is built
ros2 pkg list | grep franka_user_control
```

No joint states published

Problem: `/joint_states` topic is empty **Solution**:

```bash
# Check if fake_robot_state_publisher is running
ros2 node list | grep fake_robot_state

# Restart simulation
# Ctrl+C and relaunch
```

Commands not working

Problem: Robot doesn't respond to commands **Solution**:

```bash
# Check if simulation bridge is running
ros2 node list | grep simulation_bridge

# Verify topics are connected
ros2 topic info /velocity_command

# Check for errors in launch terminal
```

Advanced Usage

Adding Custom Simulated Behaviors

Edit `fake_robot_state_publisher.py` to add:

- Sensor noise
- Joint friction
- Gravity compensation simulation
- Custom dynamics

Recording and Playback

Record simulation data for analysis:

```bash
# Record all topics
ros2 bag record -a

# Record specific topics
ros2 bag record /joint_states /current_pose

# Playback
ros2 bag play <bag_file>
```

Integration with Other Tools

Simulation mode works with:

- RViz for visualization
- rqt for debugging
- PlotJuggler for data analysis
- Custom visualization tools

Best Practices

1. **Always test in simulation first** before running on real hardware
2. **Use conservative limits** initially
3. **Validate trajectories** for workspace violations
4. **Check velocity limits** to avoid safety reflex triggers
5. **Test error handling** with invalid commands
6. **Monitor resource usage** if running on limited hardware

Next Steps

After validating in simulation:

1. Review the Safety Guide
2. Configure your robot IP in `config/robot_config.yaml`
3. Start with conservative limits on real hardware
4. Gradually increase velocity/acceleration as confidence grows

Support

For simulation-specific issues:

- Check ROS2 topics with `ros2 topic list`
- Monitor node health with `ros2 node list`
- View logs with `ros2 run ... --ros-args --log-level debug`
- Report issues on GitHub with simulation logs